

Data Abstraction and Object Orientation (CSF301 Assignment)

Pranav M R, Arinjay Jain, Kanishk Rai, Anshul Rakesh

September 18, 2025

Contents

1	Class Hierarchy Overview	2
2	Class Hierarchy Flowchart	2
3	UML Class Diagrams	3
3.1	Stack Implementation	3
3.2	Queue Family	4
3.3	TraversableList Family	5
4	Complete Class Hierarchy Overview	7
5	Detailed Class Specifications	7
5.1	AbstractList - Root Abstract Class	7
5.2	Stack - LIFO Container	8
5.3	Queue - FIFO Container	8
5.4	PriorityQueue - Priority-Ordered Container	9
5.5	MinHeap & MaxHeap - Specialized Priority Queues	9
5.6	Deque - Double-Ended Queue	9
5.7	TraversableList - Indexable Container	10
5.8	ArrayList - Dynamic Array Implementation	10
5.9	LinkedList - Node-Based Storage	11
5.10	SinglyLinkedList - Forward-Only Linked List	11
5.11	DoublyLinkedList - Bidirectional Linked List	11
6	Performance Characteristics	12
7	Conclusion	12

1 Class Hierarchy Overview

The design follows a clear inheritance structure with **AbstractList** as the root abstract class, providing fundamental list operations. The hierarchy branches into three main categories:

1. **Stack**: Direct concrete implementation of LIFO behavior
2. **Queue**: FIFO behavior with extensions for priority and deque functionality
3. **TraversableList**: Lists that support iteration and indexing, with array and linked implementations

2 Class Hierarchy Flowchart

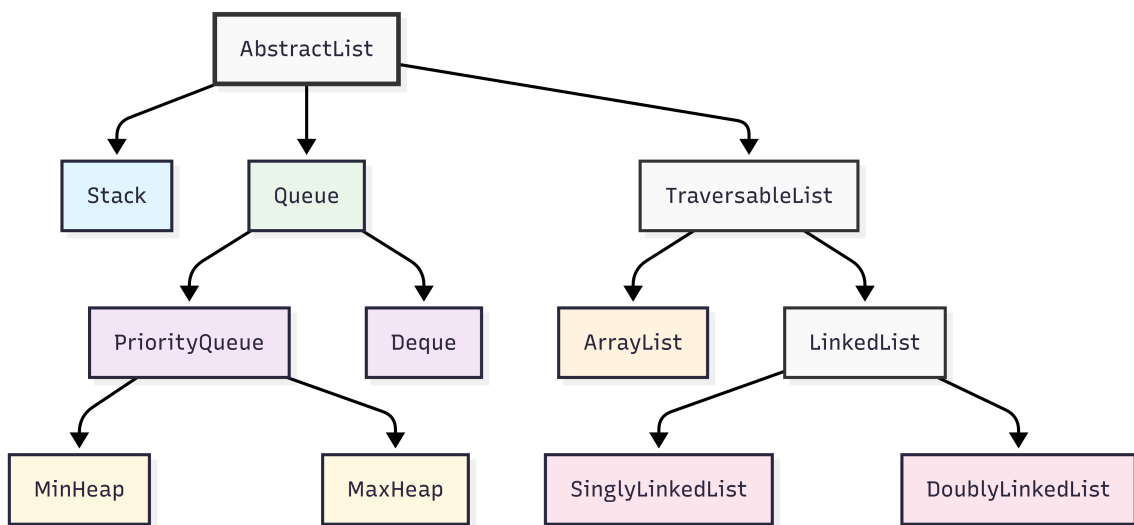


Figure 1: Complete Class Hierarchy Structure

3 UML Class Diagrams

3.1 Stack Implementation

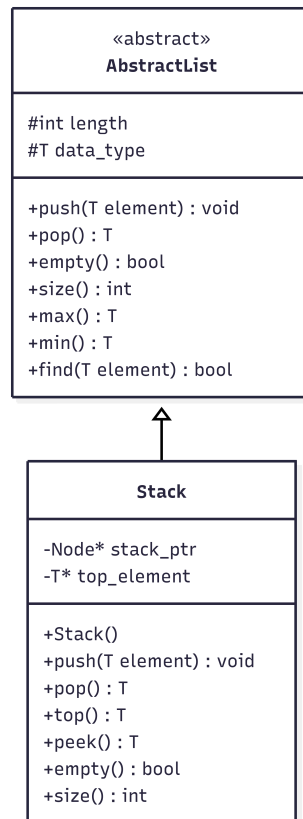


Figure 2: Stack Class Inheritance

The Stack class provides a Last-In-First-Out (LIFO) data structure implementation. It inherits from **AbstractList** and specializes the push/pop operations to work exclusively with the top of the stack.

Key Design Features:

- **Single Access Point:** All operations occur at the top of the stack
- **Node-Based Storage:** Uses linked nodes for dynamic sizing
- **Top Element Caching:** Maintains a pointer to the top element for $O(1)$ access
- **Exception Safety:** Throws underflow exceptions when popping from empty stack

Memory Management:

- Dynamic allocation/deallocation of nodes
- Automatic cleanup in destructor
- No memory leaks through proper node management

3.2 Queue Family

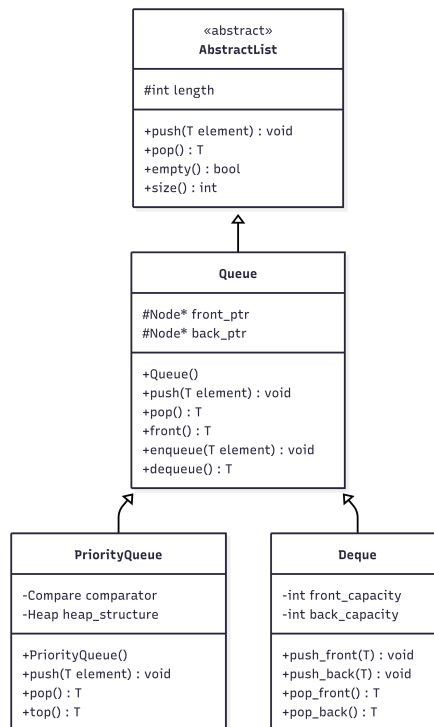


Figure 3: Queue Family Class Hierarchy

The Queue family implements First-In-First-Out (FIFO) data structures with various specializations for different use cases.

Base Queue Features:

- **Dual-Pointer Design:** Maintains separate front and back pointers for $O(1)$ operations
- **FIFO Semantics:** Elements added at back, removed from front
- **Flexible Interface:** Supports both push/pop and enqueue/dequeue terminology

PriorityQueue Specialization:

- **Heap-Based Implementation:** Uses binary heap for efficient priority ordering
- **Customizable Comparator:** Allows user-defined priority functions
- **Logarithmic Complexity:** $O(\log n)$ insertion and removal
- **Constant Top Access:** $O(1)$ access to highest priority element

Deque Capabilities:

- **Double-Ended Operations:** Insertion and removal at both ends
- **Capacity Management:** Tracks available space at front and back
- **Versatile Usage:** Can function as both stack and queue
- **Optimized Storage:** Circular buffer implementation for space efficiency

3.3 TraversableList Family

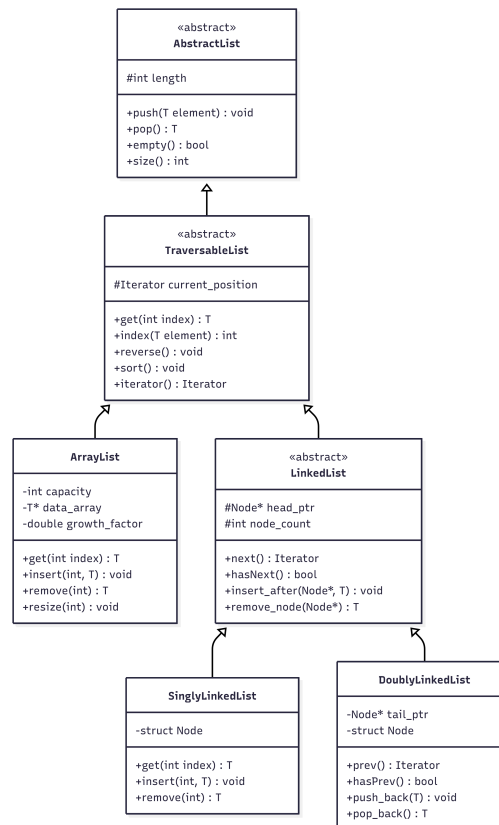


Figure 4: TraversableList Family Class Hierarchy

The TraversableList family provides indexed access and iteration capabilities, implementing two fundamental storage strategies: array-based and linked-node-based.

TraversableList Abstract Features:

- **Iterator Pattern Implementation:** Provides standard iteration interface
- **Random Access Support:** Get/set elements by index
- **Sorting Capabilities:** Built-in sort with customizable comparators
- **Reversal Operations:** In-place list reversal
- **Search Functionality:** Find elements and return indices

ArrayList Characteristics:

- **Contiguous Memory:** Array-based storage for cache efficiency
- **Dynamic Resizing:** Automatic capacity expansion with configurable growth factor
- **Random Access:** $O(1)$ indexed access to elements
- **Amortized Performance:** $O(1)$ amortized insertion at end
- **Memory Optimization:** Shrink-to-fit functionality to reduce memory footprint

LinkedList Family:

SinglyLinkedList Features:

- **Memory Efficient:** Minimal overhead per node (one pointer)
- **Forward Traversal:** Unidirectional iteration capability
- **Dynamic Size:** No pre-allocation required
- **Efficient Head Operations:** $O(1)$ insertion/deletion at beginning

DoublyLinkedList Features:

- **Bidirectional Traversal:** Forward and backward iteration
- **Tail Pointer Optimization:** $O(1)$ operations at both ends
- **Flexible Navigation:** Previous/next navigation in constant time
- **Enhanced Iterator:** Support for reverse iteration patterns

Performance Trade-offs:

Operation	ArrayList	SinglyLinked	DoublyLinked
Random Access	$O(1)$	$O(n)$	$O(n)$
Insert at Head	$O(n)$	$O(1)$	$O(1)$
Insert at Tail	$O(1)^*$	$O(n)$	$O(1)$
Insert at Middle	$O(n)$	$O(1)^{**}$	$O(1)^{**}$
Memory per Element	Low	Medium	High
Cache Performance	Excellent	Poor	Poor

Table 1: TraversableList Implementation Comparison (*Amortized, **At known position)

4 Complete Class Hierarchy Overview

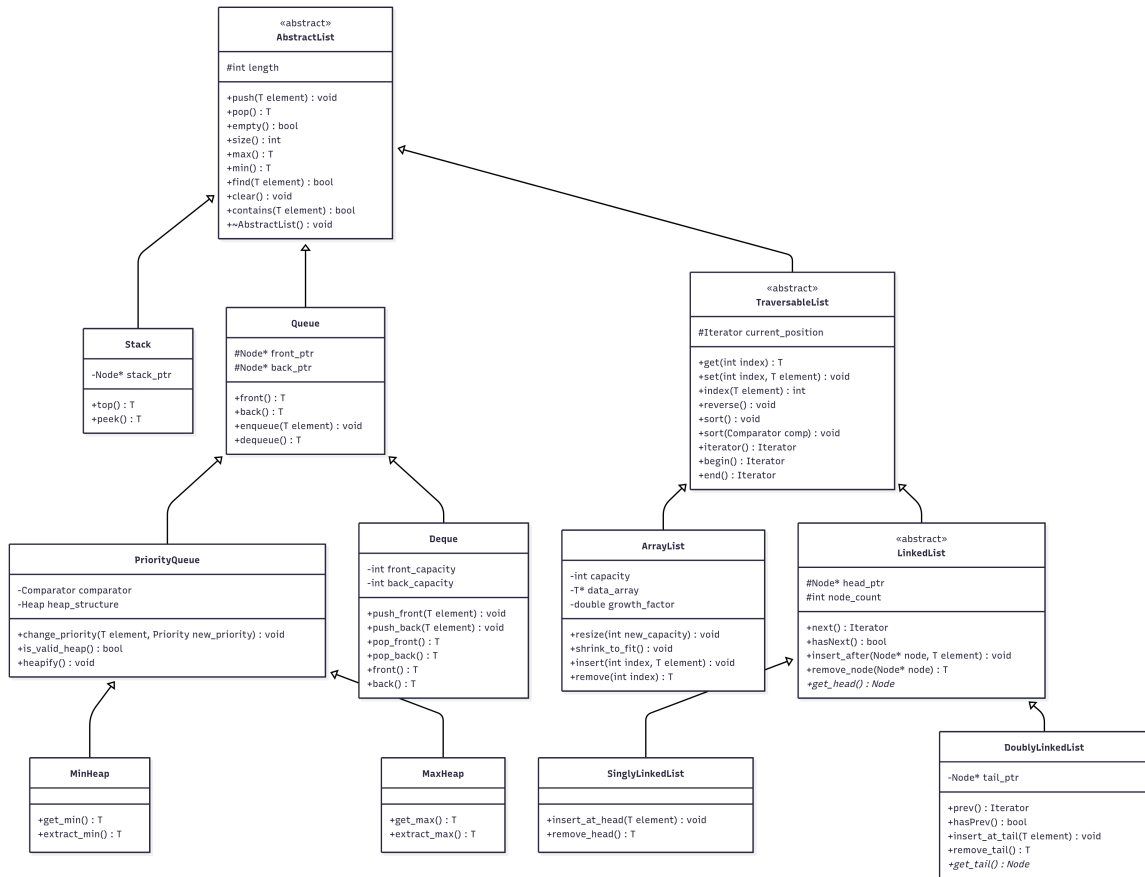


Figure 5: Complete UML Class Hierarchy with All Relationships

5 Detailed Class Specifications

5.1 AbstractList - Root Abstract Class

Purpose: Provides fundamental list operations as an interface contract.

Attributes:

- **length:** Current number of elements in the list
- **data_type:** Template parameter for element type

Key Methods and API Promises:

- **push(element):** Add element to the list (implementation varies by subclass)
- **pop():** Remove and return an element (implementation varies by subclass)
- **empty():** Returns true if list has no elements - $O(1)$
- **size():** Returns current number of elements - $O(1)$
- **max():** Returns maximum element in the list - $O(n)$
- **min():** Returns minimum element in the list - $O(n)$

- `find(element)`: Returns true if element exists - $O(n)$ typical
- `clear()`: Remove all elements - $O(n)$

5.2 Stack - LIFO Container

Purpose: Last In, First Out container with top-only access.

Additional Attributes:

- `stack_ptr`: Pointer to top of stack
- `top_element`: Cache for top element access

API Promises:

- `push(element)`: Add element to top of stack - $O(1)$
- `pop()`: Remove and return top element - $O(1)$
- `top()`: Return top element without removing - $O(1)$
- `peek()`: Alias for `top()` - $O(1)$
- Throws `std::underflow_error` on pop/top when empty
- Thread-safe operations (optional extension)

5.3 Queue - FIFO Container

Purpose: First In, First Out container with front and back access.

Additional Attributes:

- `front_ptr`: Pointer to front of queue
- `back_ptr`: Pointer to back of queue

API Promises:

- `push(element)`: Add element to back (enqueue) - $O(1)$
- `pop()`: Remove and return front element (dequeue) - $O(1)$
- `front()`: Return front element without removing - $O(1)$
- `enqueue(element)`: Alias for `push()` - $O(1)$
- `dequeue()`: Alias for `pop()` - $O(1)$
- Throws `std::underflow_error` on dequeue/front when empty

5.4 PriorityQueue - Priority-Ordered Container

Purpose: Queue where elements are served based on priority rather than insertion order.

Additional Attributes:

- `comparator`: Function object for element comparison
- `heap_structure`: Internal heap for maintaining priority order

API Promises:

- `push(element)`: Insert maintaining priority order - $O(\log n)$
- `pop()`: Remove and return highest priority element - $O(\log n)$
- `top()`: Return highest priority element - $O(1)$
- `change_priority(element, new_priority)`: Update priority - $O(\log n)$
- Customizable comparison function
- Stable ordering for equal priorities (optional)

5.5 MinHeap & MaxHeap - Specialized Priority Queues

Purpose: Pre-configured priority queues for common use cases.

MinHeap API Promises:

- Smallest element has highest priority
- `get_min()`: Return minimum element - $O(1)$
- All PriorityQueue operations with min-comparator

MaxHeap API Promises:

- Largest element has highest priority
- `get_max()`: Return maximum element - $O(1)$
- All PriorityQueue operations with max-comparator

5.6 Deque - Double-Ended Queue

Purpose: Queue allowing efficient insertion and removal at both ends.

Additional Attributes:

- `front_capacity`: Available space at front
- `back_capacity`: Available space at back

API Promises:

- `push_front(element)`: Add element to front - $O(1)$
- `push_back(element)`: Add element to back - $O(1)$
- `pop_front()`: Remove from front - $O(1)$

- `pop_back()`: Remove from back - $O(1)$
- `front()`: Access front element - $O(1)$
- `back()`: Access back element - $O(1)$
- Can function as both stack and queue

5.7 TraversableList - Indexable Container

Purpose: Abstract base for lists supporting indexed access and iteration.

Additional Attributes:

- `current_position`: Iterator position for traversal

API Promises:

- `get(index)`: Return element at specified index
- `index(element)`: Return index of first occurrence
- `reverse()`: Reverse the order of elements
- `sort()`: Sort elements in ascending order
- `sort(comparator)`: Sort with custom comparator
- `iterator()`: Return iterator for traversal
- `begin()/end()`: Standard iterator interface
- Support for range-based for loops

5.8 ArrayList - Dynamic Array Implementation

Purpose: Resizable array with random access capabilities.

Additional Attributes:

- `capacity`: Maximum elements before reallocation
- `data_array`: Dynamic array for element storage
- `growth_factor`: Capacity expansion multiplier (default 2.0)

API Promises:

- `get(index)`: Direct array access - $O(1)$
- `push(element)`: Add to end, resize if needed - Amortized $O(1)$
- `insert(index, element)`: Insert at position - $O(n)$
- `remove(index)`: Remove element at index - $O(n)$
- `resize(new_capacity)`: Change array capacity
- `shrink_to_fit()`: Reduce capacity to match size
- Automatic capacity management
- Strong exception safety for all operations

5.9 LinkedList - Node-Based Storage

Purpose: Abstract base for linked storage implementations.

Additional Attributes:

- `head_ptr`: Pointer to first node
- `node_count`: Number of nodes

API Promises:

- `next()`: Move to next element in iteration
- `hasNext()`: Check if next element exists
- `insert_after(node, element)`: Insert after given node - $O(1)$
- `remove_node(node)`: Remove specific node - $O(1)$
- Dynamic size without pre-allocation
- Efficient insertion/deletion at known positions

5.10 SinglyLinkedList - Forward-Only Linked List

Purpose: Memory-efficient linked list with forward traversal.

Node Structure:

```
1 struct Node {  
2     T data;  
3     Node* next_ptr;  
4 };
```

API Promises:

- `get(index)`: Sequential search from head - $O(n)$
- `insert(index, element)`: Insert at position - $O(n)$
- `remove(index)`: Remove at position - $O(n)$
- Minimal memory overhead per node
- $O(1)$ insertion/deletion at head
- Forward iteration only

5.11 DoublyLinkedList - Bidirectional Linked List

Purpose: Linked list supporting efficient bidirectional traversal.

Additional Attributes:

- `tail_ptr`: Pointer to last node

Node Structure:

```

1 struct Node {
2     T data;
3     Node* next_ptr;
4     Node* prev_ptr;
5 };

```

API Promises:

- `prev()`: Move to previous element - $O(1)$
- `hasPrev()`: Check if previous element exists - $O(1)$
- `push_back(element)`: Add to tail - $O(1)$
- `pop_back()`: Remove from tail - $O(1)$
- Bidirectional traversal capability
- $O(1)$ insertion/deletion at both ends
- $O(1)$ insertion/deletion at known positions

6 Performance Characteristics

Operation	Stack	Queue	ArrayList	LinkedList
Access by Index	N/A	N/A	$O(1)$	$O(n)$
Search	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Insert at End	$O(1)$	$O(1)$	$O(1)^*$	$O(1)$
Insert at Beginning	N/A	N/A	$O(n)$	$O(1)$
Insert at Middle	N/A	N/A	$O(n)$	$O(1)^{**}$
Delete at End	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Delete at Beginning	N/A	N/A	$O(n)$	$O(1)$
Delete at Middle	N/A	N/A	$O(n)$	$O(1)^{**}$

Table 2: Time Complexity Comparison (* Amortized, ** At known position)

7 Conclusion

This comprehensive class hierarchy design provides a flexible, extensible framework for various container types while maintaining clear inheritance relationships and well-defined APIs. The design emphasizes performance, memory efficiency, and type safety while following established object-oriented design principles.